

BILP, Quasi-symmetric Method

Matlab code 사용 매뉴얼

우주비행제어 연구실
석사 5학기 이진진

Quasi-Symmetric Method

1-1. 기준 위성(Seed Satellite)의 관측시간 및 관심지역의 접근 정보를 정의

```
편집기 - C:\Users\ACL2022_13\Desktop\5월\Quasi-Symmetric.m
Quasi-Symmetric.m x +
1      clc
2      clear all
3
4      trials = 0;
5      start_time = tic;
6      L = 720; 전체 관측시간의 길이 (L 한개 당 2분씩, 총 1440분 = 하루)
7
8      while true
9
10     % Seoul
11
12     ff_00 = ... Freeflyer를 통해 얻은 위성 접근 시간의 Start, End time
13     [...]
14     36399.19922 37581.73828 ;
15     43423.41797 44669.47266 ;
16     50630.03906 51286.93359 ;
17     70706.30859 71445.05859 ;
18     77369.23828 78631.64063 ;
19     84492.42188 85639.27734 ;
20     ...
21     ];
22
23     % ff_0 -> n 으로 변환
24     ff_0 = floor(ff_00/(86400/L)); 시간차원에서 n차원으로 변환
25
26     % 구간의 시작점과 끝점을 변수에 저장
27     start_points = ff_0(:, 1);
28     end_points = ff_0(:, 2);
```

0초를 Epoch Time으로 설정했을 때, 순서대로
관심지역에 위성이 연결되는 시각, 빠져나가는 시각
(ex. 관심지역을 총 6번 지남)

관심지역을 지나는 모든 시작 지점과 종료 지점을 저장

1-2. 주석 및 주석해제 : 필요한 위성 수만큼 주석 해제하여 위성 수 설정

```
편집기 - C:\Users\ACL2022_13\Desktop\5월\Quasi_Symmetric.m
Quasi_Symmetric.m x +
31     %% Random (ex. 위성 22대)
32
33     integer_intervals_1 = [];
34     integer_intervals_2 = [];
35     integer_intervals_3 = [];
36     integer_intervals_4 = [];
37     integer_intervals_5 = [];
38     integer_intervals_6 = [];
39     integer_intervals_7 = [];
40     integer_intervals_8 = [];
41     integer_intervals_9 = [];
42     integer_intervals_10 = [];
43     integer_intervals_11 = [];
44     integer_intervals_12 = [];
45     integer_intervals_13 = [];
46     integer_intervals_14 = [];
47     integer_intervals_15 = [];
48     integer_intervals_16 = [];
49     integer_intervals_17 = [];
50     integer_intervals_18 = [];
51     integer_intervals_19 = [];
52     integer_intervals_20 = [];
53     integer_intervals_21 = [];
54     integer_intervals_22 = [];
55     % integer_intervals_23 = [];
```

필요한 총 위성의 수 N에 따라 integer_intervals_N만큼
빈 행렬을 생성 (예시로 22대로 설정)

사용자가 임의로 위성 수를 추가 또는 감소 가능

1-3. 주석 해제한 Line의 수만큼 위성의 수가 정의되도록 vars 함수를 사용

```
편집기 - C:\Users\ACL2022_13\Desktop\5월\Quasi_Symmetric.m
Quasi_Symmetric.m x +
71 % 변수 목록
72 vars = who;
73
74 % 'integer_intervals_'로 시작하는 변수들만 필터링
75 pattern = '^integer_intervals_\d+$'; 위에서 설정한 빈 행렬의 수만큼 위성의 수 N을 결정
76
77 N = 0; N을 0부터 시작하여
78
79 for i = 1:length(vars)
80     if ~isempty(regexpi(vars{i}, pattern, 'once'))
81         N = N + 1; 'integer_intervals_'로 시작하고 그 뒤에 숫자가 오는 변수를 찾음
82     end
83     해당하는 변수가 있으면 1을 반복하여 증가시키며 몇 개 있는지 계산
```

2-1. 기준 위성을 첫 번째 위성으로 정의하고 기준 위성의 위치를 바탕으로 학위논문의 식 4.1에 따라 모든 위성의 위치(Ω, M)를 상대적으로 결정

```
편집기 - C:\Users\ACL2022_13\Desktop\5월\Quasi_Symmetric.m *
Quasi_Symmetric.m * x +
85 for i = 1:length(ff_0)
86     % satellite 1 위치 (Seed Satellite) 기준 위성을 시작으로 위성들의 상대적 위치를 배치
87     interval_1 = start_points(i):end_points(i);
88     interval_1(interval_1 > (L-1)) = interval_1(interval_1 > (L-1)) - L;
89     integer_intervals_1 = [integer_intervals_1, interval_1];
90
91     전체 관측시간의 길이를 초과할 경우
92     0부터 다시 반복되도록 보정 (720은 0으로)
93
94     % satellite 2 위치 기준 위성을 중심으로 전지구에 균등하게 배치
95     interval_2 = [interval_1 + round(L/N)]; (학위논문 20쪽 4.1식)
96     interval_2(interval_2 > (L-1)) = interval_2(interval_2 > (L-1)) - L;
97     integer_intervals_2 = [integer_intervals_2, interval_2];
98
99     % satellite 3 위치
100    interval_3 = [interval_1 + round(L/N*(N-1))];
101    interval_3(interval_3 > (L-1)) = interval_3(interval_3 > (L-1)) - L;
102    integer_intervals_3 = [integer_intervals_3, interval_3];
103
104    % satellite 4 위치
105    interval_4 = [interval_1 + round(L/N*(N-2))];
106    interval_4(interval_4 > (L-1)) = interval_4(interval_4 > (L-1)) - L;
107    integer_intervals_4 = [integer_intervals_4, interval_4];
108
109    :
110
111    % satellite 22 위치 필요한 총 위성 수(ex. 22대) 만큼 반복
112    interval_22 = [interval_1 + round(L/N*(N-20))];
113    interval_22(interval_22 > (L-1)) = interval_22(interval_22 > (L-1)) - L;
114    integer_intervals_22 = [integer_intervals_22, interval_22];
115
116    % % satellite 23 위치
117    % interval_23 = interval_1 + round(L/N*(N-21));
118    % interval_23(interval_23 > (L-1)) = interval_23(interval_23 > (L-1)) - L;
119    % integer_intervals_23 = [integer_intervals_23, interval_23];
```

2-2. 모든 위성이 관심지역을 지나는 정보를 누적하여 합산함

```
편집기 - C:\Users\ACL2022_13\Desktop\#5월\Quasi_Symmetric.m
Quasi_Symmetric.m x +
238 sum_intervals = ...
239     [integer_intervals_1, ...
240     integer_intervals_2, ...
241     integer_intervals_3, ...
242     integer_intervals_4, ...
243     integer_intervals_5, ...
244     integer_intervals_6, ...
245     integer_intervals_7, ...
246     integer_intervals_8, ...
247     integer_intervals_9, ...
248     integer_intervals_10, ...
249     integer_intervals_11, ...
250     integer_intervals_12, ...
251     integer_intervals_13, ...
252     integer_intervals_14, ...
253     integer_intervals_15, ...
254     integer_intervals_16, ...
255     integer_intervals_17, ...
256     integer_intervals_18, ...
257     integer_intervals_19, ...
258     integer_intervals_20, ...
259     integer_intervals_21, ...
260     integer_intervals_22, ...
261     % integer_intervals_23, ...
```

위에서 정의된 모든 위성이 지나는
Timeline의 정보를 합산함

2-3. 전체 관측 시간 중, n의 값에 따른 위성의 방문 횟수를 설정

```
편집기 - C:\Users\ACL2022_13\Desktop\#5월\Quasi_Symmetric.m
Quasi_Symmetric.m x +
271 unique_intervals = unique(sort(sum_intervals)); 누적된 정보 중, 중복되는 n의 값을 제거
272
273 % 최소 및 최대 값 계산
274 min_val = min(sum_intervals);
275 max_val = max(sum_intervals); (모든)위성이 관심지역에 접근하는 n의 최소, 최대 값 설정
276
277 % 모든 숫자에 대해 개수를 기록할 행렬 초기화
278 sum_intervals_ac = zeros(L, 2); L행 2열의 빈 행렬을 생성
279
280 % 1열에 숫자들 채우기
281 sum_intervals_ac(:, 1) = (0:L-1)'; 모든 n의 값(0~719의 정수)을 1열에 입력
282
283 % 각 숫자의 개수 세기
284 for i = min_val:max_val 각 n에 대해 위성이 관심지역을 방문한 횟수를 2열에 입력
285     sum_intervals_ac(i - min_val + 1, 2) = sum(sum_intervals == i);
286 end
287
288
289 Cov = length(unique_intervals) / L * 100 전체 관측 시간 동안 위성이 관심지역을
290                                     통과하는 커버리지 비율을 계산
291 if Cov >= 100
292     break; 요구조건을 값을 입력하여 사용자가 설정한 위성 수로
293     end 원하는 커버리지 비율을 달성 가능 여부 확인 가능
294     trials = trials + 1;
295 end
296
297
```


3. 그래프 기본 정보 설정 : 모든 위성의 시작, 종료시각 저장 및 전체 관측시간의 총 길이 L을 초과하는 경우의 보정

```

307 %% 그래프
308
309 n_1 = [round(L/N * (0:1:N-1))]+1; % 학위논문 20쪽, 식 4.1에 의해 계산된
% 모든 위성의 n의 배열을 n_1에 저장
310
311 % 배열 인덱스는 양의 정수만 가능하므로 모든
% n의 값에 1을 더함(상대 위치는 변함 없음)
312 % ff_ 변수를 생성 및 할당
313 for i = 1:N
314     eval(['ff_', num2str(i), ' = ff_0 + n_1(', num2str(i), ');']);
315 end
316 % 기준 위성이 관심지역을 지나는 시작 및 종료 지점에 n의 배열을 더함
317
318 % x축 범위 설정
319 x_range = 0:L-1;
320
321 for idx = 1:length(n_1)
322     ff = eval(['ff_', num2str(idx)]);
323     x_coords1{idx} = []; % 해당 작업공간의 x좌표를 저장할 배열 초기화
324
325     for i = 1:size(ff,1)
326         start_time = ff(i, 1); % 모든 위성이 관심지역을 지나는
327         end_time = ff(i, 2); % 시작 및 종료 지점을 n 값으로 저장 (f_1 ~ f_22)
328
329         % 관측 시간의 총 길이에 따른 보정
330         if end_time > L-1 && start_time < L
331             start_index = find(x_range >= start_time, 1);
332             end_index = find(x_range < end_time-L, 1, 'last');
333             % 시작과 종료 시각 사이에
334             % 관측 시간의 총 길이가
335             % 포함되는 경우
336         elseif start_time > L-1
337             start_index = find(x_range >= start_time-L, 1);
338             end_index = find(x_range <= end_time-L, 1, 'last');
339             % 관측 시간의 총 길이를
340             % 초과하는 경우
341         else
342             start_index = find(x_range >= start_time, 1);
343             end_index = find(x_range <= end_time, 1, 'last');
344             % 일반적인 경우
345         end
346
347         % 시작 시간부터 종료 시간까지의 x좌표를 배열에 저장
348         if end_time > L-1 && start_time < L
349             x_coords1{idx} = [x_coords1{idx}, x_range(start_index:L),(0:end_index)];
350
351         else
352             x_coords1{idx} = [x_coords1{idx}, x_range(start_index:end_index)];
353         end
354     end
355 end
356 end

```

4-1. 기준 위성의 Access Profile 그래프 정보 설정

```
편집기 - C:\Users\ACL2022_13\Desktop\#5\Quasi_Symmetric.m *
Quasi_Symmetric.m * +
354 %% Access Profile
355
356 % x축 범위 설정
357 x_range = 0:L-1;
358
359 figure;
360 subplot(3, 1, 1);
361
362 hex_color1 = '#76A646'; RGB HEX코드를 이용하여 그래프의 색상 정의
363
364 red1 = hex2dec(hex_color1(2:3)) / 255; % R 값
365 green1 = hex2dec(hex_color1(4:5)) / 255; % G 값
366 blue1 = hex2dec(hex_color1(6:7)) / 255; % B 값
367
368 hex_color11 = '#76A646'; RGB HEX코드를 이용하여 그래프의 색상 정의
369
370 red11 = hex2dec(hex_color11(2:3)) / 255; % R 값
371 green11 = hex2dec(hex_color11(4:5)) / 255; % G 값
372 blue11 = hex2dec(hex_color11(6:7)) / 255; % B 값
373
374
375 % 시작 시간과 종료 시간에 해당하는 x 인덱스 찾기
376 start_index = find(x_range >= start_time, 1);
377 end_index = find(x_range <= end_time, 1, 'last');
378
379 % y 값 설정
380 y_values1a = zeros(size(x_range)); 위성이 관심지역을 통과하는 시작 및 종료
381 y_values1a(start_index:end_index) = 1; 시간에만 그래프에 출력하도록 설정
382
383 % 그래프 그리기
384 plot(x_range, y_values1a, '-o r', 'LineWidth', 1, 'MarkerSize', 1, 'Color', [red1, green1, blue1]);
385 hold on;
386 area(x_range, y_values1a, 'FaceColor', [red11, green11, blue11]);
387 plot(x_range, y_values1a, 'Color', [red1, green1, blue1], 'LineWidth', 1.5);
388 end 선의 색상, 스타일 및 굵기 등 그래프 세부 설정
389
390 % 그래프 세부 설정
391 xlabel('n');
392 ylabel('v0j[n]');
393 title('Access Profile');
394 grid on;
395 xlim([0, L-1]);
```

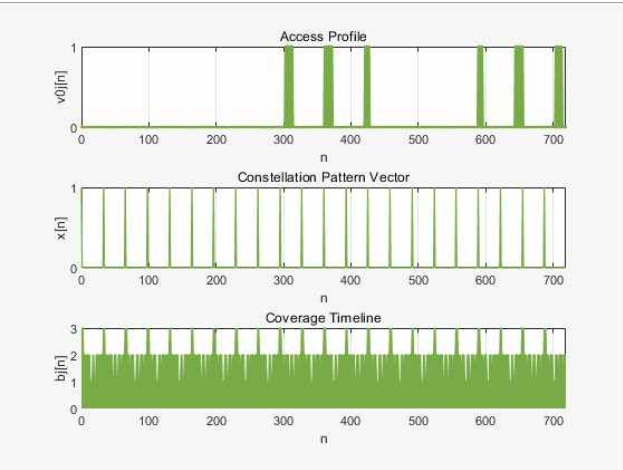
4-2. Constellation Pattern Vector 그래프 정보 설정

```
편집기 - C:\Users\WACL2022_13\Desktop\#5\Quasi_Symmetric.m
Quasi_Symmetric.m
405 %% Constellation Pattern Vector
406
407 % x_range = 0:L-1;
408
409 subplot(3, 1, 2);
410
411 % y 값 설정
412 y_values1p = zeros(size(x_range)); x범위의 모든 값을 0으로 초기화 한 후에
413 y_values1p(n_1(1,:)) = 1; n의 배열에 해당하는 값만 1로 설정
414
415 plot(x_range, y_values1p, 'Color', [red1, green1, blue1], 'LineWidth', 1, 'MarkerSize', 1);
416 title('Constellation Pattern Vector');
417 hold on;
418
419 xlim([0, L-1]);
420 ylim([0, 1]);
421 yticks([0 1]);
422
423 % 그래프 세부 설정
424 xlabel('n');
425 ylabel('x[n]');
426 grid on;
427 xlim([0, L-1]);
428 ylim([0, 1]);
429 yticks([0 1]);
430
```

4-3. 모든 위성의 Coverage Timeline 그래프 정보 설정

```
편집기 - C:\Users\WACL2022_13\Desktop\#5\Quasi_Symmetric.m
Quasi_Symmetric.m
431 %% Coverage Timeline
432
433 x_range = 0:L-1;
434
435 subplot(3, 1, 3); Line 277~286 에서 설정한 모든 위성의 방문 횟수 정보를 불러옴
436 plot(sum_intervals_ac(:,1), sum_intervals_ac(:,2), 'Color', [red1, green1, blue1], 'LineWidth', 1, 'Mark
437 title('Coverage Timeline');
438 hold on;
439 area(sum_intervals_ac(:,1), sum_intervals_ac(:,2), 'FaceColor', [red11, green11, blue11]);
440 plot(sum_intervals_ac(:,1), sum_intervals_ac(:,2), 'Color', [red1, green1, blue1], 'LineWidth', 1);
441
442 xlim([0, L-1]);
443 ylim([0, 3]);
444
445 % 그래프 세부 설정
446 xlabel('n');
447 ylabel('bj[n]');
448 grid on;
449 xlim([0, L-1]);
450
```

그래프 결과



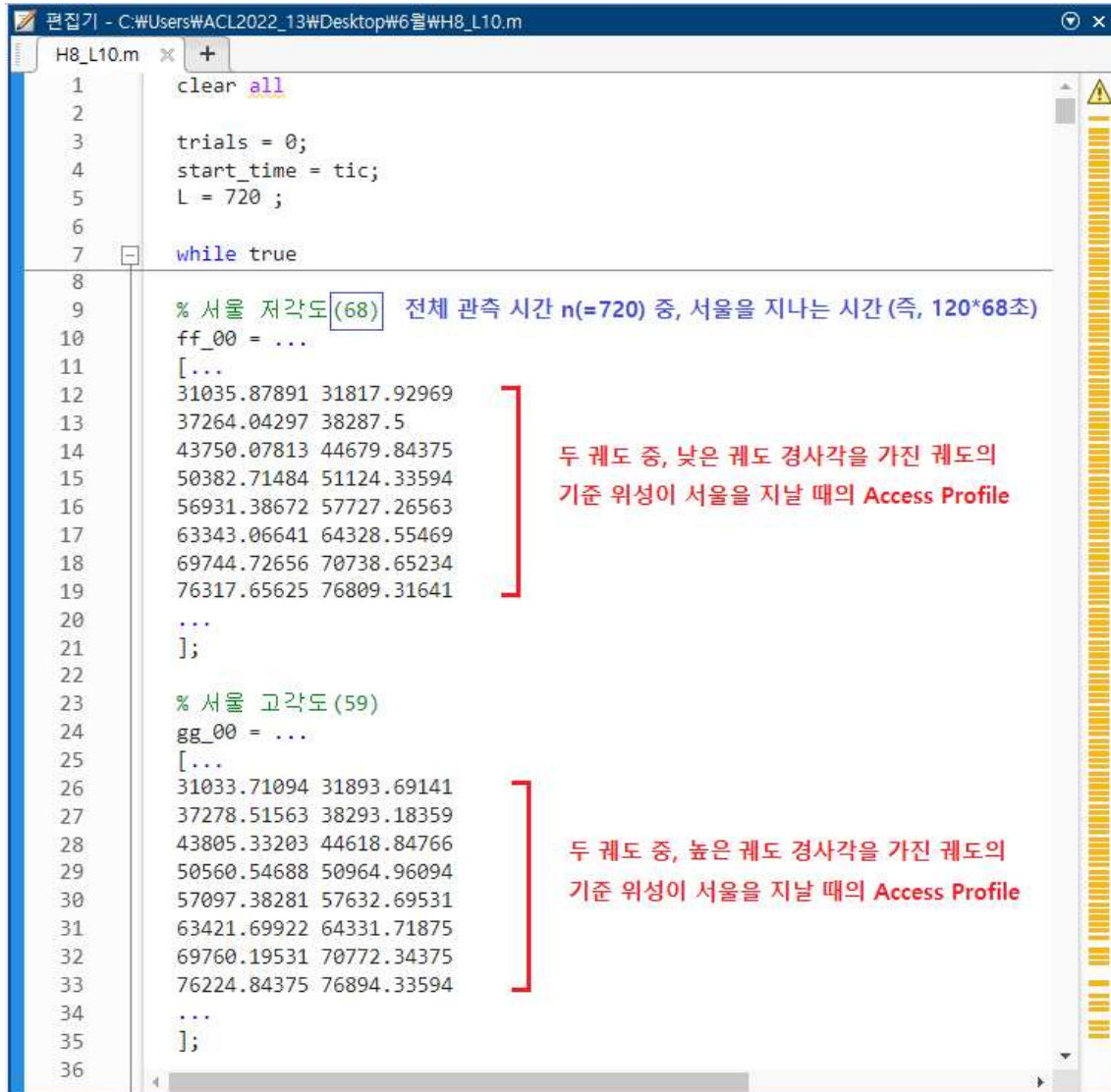
전체 관측 시간($n : 0 \sim 719$ 까지의 정수, $720 \times 120s = 86400$ 초) 중,
기준 위성 한 대가 관심지역을 지나는 구간에 대한 정보

학위논문 20쪽, 식 4.1에 따라 필요한 총 위성을 전체 관측 시간동안
균등하게 배치하였을 때의 n 의 정보(모든 위성의 상대적 위치)

전체 관측 시간동안 n 에 따른 관심지역을 지나는 모든 위성의 방문
횟수

BILP Method

1-1. 기준 위성(Seed Satellite)의 관측시간 및 관심지역의 접근 정보를 정의



```
1 clear all
2
3 trials = 0;
4 start_time = tic;
5 L = 720 ;
6
7 while true
8
9     % 서울 저각도 (68) 전체 관측 시간 n(=720) 중, 서울을 지나는 시간 (즉, 120*68초)
10    ff_00 = ...
11    [...
12    31035.87891 31817.92969
13    37264.04297 38287.5
14    43750.07813 44679.84375
15    50382.71484 51124.33594
16    56931.38672 57727.26563
17    63343.06641 64328.55469
18    69744.72656 70738.65234
19    76317.65625 76809.31641
20    ...
21    ];
22
23    % 서울 고각도 (59)
24    gg_00 = ...
25    [...
26    31033.71094 31893.69141
27    37278.51563 38293.18359
28    43805.33203 44618.84766
29    50560.54688 50964.96094
30    57097.38281 57632.69531
31    63421.69922 64331.71875
32    69760.19531 70772.34375
33    76224.84375 76894.33594
34    ...
35    ];
36
```

두 궤도 중, 낮은 궤도 경사각을 가진 궤도의
기준 위성이 서울을 지날 때의 Access Profile

두 궤도 중, 높은 궤도 경사각을 가진 궤도의
기준 위성이 서울을 지날 때의 Access Profile

```
편집기 - C:\Users\ACL2022_13\Desktop\#6\H8_L10.m
H8_L10.m x +
37 % 블라디 저각도 (62)
38 hh_00 = ...
39 [...]
40 31136.19141 31955.21484
41 37381.52344 38405.09766
42 43841.30859 44813.20313
43 50394.72656 51281.07422
44 56890.3125 57831.50391
45 63306.67969 64331.71875
46 69730.19531 70650.46875
47 ...
48 ];
49
50 % 블라디 고각도 (65)
51 ii_00 = ...
52 [...]
53 31128.57422 32026.46484
54 37395.76172 38412.30469
55 43892.46094 44769.02344
56 50501.42578 51205.95703
57 56988.04688 57799.92188
58 63355.54688 64349.47266
59 69721.17188 70694.35547
60 76263.33984 76675.78125
61 ...
62 ];
63
64
65 % ff_0 -> n 으로 변환
66 ff_0 = floor(ff_00/(86400/L));
67 neg1 = ff_0;
68
69 gg_0 = floor(gg_00/(86400/L));
70 neg2 = gg_0;
71
72 hh_0 = floor(hh_00/(86400/L));
73 neg3 = hh_0;
74
75 ii_0 = floor(ii_00/(86400/L));
76 neg4 = ii_0;
77
78
79
80 % 구간의 시작점과 끝점을 변수에 저장
81 start_points1 = ff_0(:, 1);
82 end_points1 = ff_0(:, 2);
83
84 start_points2 = gg_0(:, 1);
85 end_points2 = gg_0(:, 2);
86
87 start_points3 = hh_0(:, 1);
88 end_points3 = hh_0(:, 2);
89
90 start_points4 = ii_0(:, 1);
91 end_points4 = ii_0(:, 2);
```

두 궤도 중, 낮은 궤도 경사각을 가진 궤도의
기준 위성이 블라디보스토크를 지날 때의
Access Profile

두 궤도 중, 높은 궤도 경사각을 가진 궤도의
기준 위성이 블라디보스토크를 지날 때의
Access Profile

시간 차원에서 n차원으로 변환

관심지역을 지나는 모든 시작 지점과
종료 지점을 저장

2-1. 높은 경사각의 궤도를 가진 위성부터 배치 준비

```

편집기 - C:\Users\ACL2022_13\Desktop\#6월\H8_L10.m *
H8_L10.m * x +
94 % 모든 구간을 정수로 표시해서 작업 공간에 저장
95 result_kh1 = [];
96 result_kh2 = [];
97 result_kh3 = [];
98 result_kh4 = [];
99 result_kh5 = [];
100 result_kh6 = [];
101 result_kh7 = [];
102 result_kh8 = [];
103
104 result_eh1 = [];
105 result_eh2 = [];
106 result_eh3 = [];
107 result_eh4 = [];
108 result_eh5 = [];
109 result_eh6 = [];
110 result_eh7 = [];
111 result_eh8 = [];

```

위성에 의한 커버리지 결과 출력을 위해 빈 행렬을 생성

높은 경사각의 궤도를 가진 위성이
서울을 지나는 커버리지 결과

높은 경사각의 궤도를 가진 위성이
블라디보스토크를 지나는 커버리지 결과

2-2. 기준 위성을 중심으로 위성 배치 시작(서울)

```

편집기 - C:\Users\ACL2022_13\Desktop\#6월\H8_L10.m
H8_L10.m x +
139 %% K_Satellite Setting(H2 / L0) 기준 위성을 바탕으로 서울을 지나는 위성 배치
140 (높은 경사각 궤도: 2대 / 낮은 경사각 궤도: 0대)
141 for n = 0:(L-1)
142 integer_intervals_kh1 = [];
143 integer_intervals_kh2 = [];
144
145
146 for i = 1:length(gg_0)
147 % 고각도 satellite 1 위치 생성 첫 번째 위성은 기준 위성으로 설정
148 interval_kh1 = start_points2(i) : end_points2(i);
149 interval_kh1(interval_kh1 > (L-1)) = interval_kh1(interval_kh1 > (L-1)) - L;
150 integer_intervals_kh1 = [integer_intervals_kh1, interval_kh1];
151
152 전체 관측 시간의 길이를 초과할 경우
153 0부터 다시 반복되도록 보정
154
155 % 고각도 satellite 2 랜덤 위치 생성
156 interval_kh2 = interval_kh1 + n; 두 번째 위성은 첫 번째 위성 기준으로 n만큼 이동한 위치에 배치
157 interval_kh2(interval_kh2 > (L-1)) = interval_kh2(interval_kh2 > (L-1)) - L;
158 integer_intervals_kh2 = [integer_intervals_kh2, interval_kh2];
159 end
160
161 sum_intervals_kh = ...
162 [integer_intervals_kh1, ...
163 integer_intervals_kh2, ...
164 ];
165 unique_intervals_kh = unique(sort(sum_intervals_kh)); n의 모든 값을 저장(중복이 포함됨)
166 중복을 제거한 값
167
168 difference_kh = length(sum_intervals_kh) - length(unique_intervals_kh);
169
170 중복된 n의 개수
171 result_kh1 = [result_kh1; n, difference_kh];
172 end
173
174 Cov_kh1 = length(integer_intervals_kh1)/L*100; 첫 번째 위성만으로 얻을 수 있는
전체 관측 시간 동안의 커버리지 비율

```

첫 번째, 두 번째 위성에 의한 Coverage를 합산함

첫 번째, 두 번째 위성에 의해 관심 지역을 지난

첫 번째 위성만으로 얻을 수 있는
전체 관측 시간 동안의 커버리지 비율

2-3. 기준 위성을 중심으로 위성 배치 시작(블라디보스토크)

```

176 % E_Satellite Setting(H2 / L0) - 기준 위성의 위치를 바탕으로 고각도 위성 1대 배치(총 2대)
177
178 for n = 0:(L-1)
179     integer_intervals_eh1 = [];
180     integer_intervals_eh2 = [];
181
182
183     for i = 1:length(ii_0)
184         % 고각도 satellite 1 위치 생성 첫 번째 위성은 기준 위성으로 설정
185         interval_eh1 = start_points4(i) : end_points4(i) ;
186         interval_eh1(interval_eh1 > (L-1)) = interval_eh1(interval_eh1 > (L-1)) - L;
187         integer_intervals_eh1 = [integer_intervals_eh1, interval_eh1];
188
189         % 고각도 satellite 2 랜덤 위치 생성
190         interval_eh2 = interval_eh1 + n;
191         interval_eh2(interval_eh2 > (L-1)) = interval_eh2(interval_eh2 > (L-1)) - L;
192         integer_intervals_eh2 = [integer_intervals_eh2, interval_eh2];
193     end
194
195     sum_intervals_eh = ...
196     [integer_intervals_eh1, ...
197     integer_intervals_eh2, ...
198     ];
199
200     unique_intervals_eh = unique(sort(sum_intervals_eh));
201     difference_eh = length(sum_intervals_eh) - length(unique_intervals_eh);
202     result_eh1 = [result_eh1; n, difference_eh];
203
204     end
205
206     Cov_eh1 = length(integer_intervals_eh1)/L*100;
207
208
209
210
211

```

기준 위성을 바탕으로 블라디보스토크를 지나는 위성 배치
(높은 경사각 궤도 : 2대 / 낮은 경사각 궤도 : 0대)

첫 번째 위성은 기준 위성으로 설정

두 번째 위성은 첫 번째 위성 기준으로 n만큼 이동한 위치에 배치

전체 관측 시간의 길이를 초과할 경우
0부터 다시 반복되도록 보정

첫 번째, 두 번째 위성에 의한 Coverage를 합산함

중복을 제거한 값

첫 번째, 두 번째 위성에 의해 관심지역을 지나간
n의 모든 값을 저장(중복이 포함됨)

중복된 n의 개수

첫 번째 위성만으로 얻을 수 있는
전체 관측 시간 동안의 커버리지 비율(블라디보스토크)

2-4. 두 관심지역에 대해 공통으로 중복을 최소화하는 random_number를 찾음

```

212 % 공통
213
214 result_sum1 = [result_kh1(:,1), result_kh1(:,2) + result_eh1(:,2)];
215
216
217 rows_with_min_sum1 = result_sum1(result_sum1(:, 2) == min(result_sum1(:, 2)), :);
218
219 k_sum1 = rows_with_min_sum1(:,1);
220
221 random_index_1 = randi([1, length(k_sum1)]);
222
223 random_number_1 = k_sum1(random_index_1);
224
225

```

서울과 블라디보스토크의 공통된 n의 중복 값을 합산함

두 지역에 동시에 중복되는 값이 최소인 n을 찾음

그 중 무작위로 하나를 선택함

(random_index_1)번째에 있는 값이 random_number_1이 됨

2-5. 추출한 random_number로 두 번째 위성의 상대위치 결정(서울)

```

225 %% K_renewal 2
226 integer_intervals_kh1 = []; 2-2 에서 진행한 것과 동일하게 반복
227 integer_intervals_kh2 = [];
228
229 for i = 1:length(gg_0)
230 % 고각도 satellite 1 위치 생성
231 interval_kh1 = start_points2(i) :end_points2(i) ;
232 interval_kh1(interval_kh1 > (L-1)) = interval_kh1(interval_kh1 > (L-1)) - L;
233 integer_intervals_kh1 = [integer_intervals_kh1, interval_kh1];
234
235 % 고각도 satellite 2 위치 생성 2-4 에서 추출한 random_number를 추가하여 두 번째 위성의 위치를 결정
236 interval_kh2 = interval_kh1 + random_number_1;
237 interval_kh2(interval_kh2 > (L-1)) = interval_kh2(interval_kh2 > (L-1)) - L;
238 integer_intervals_kh2 = [integer_intervals_kh2, interval_kh2];
239 end
240
241
242 sum_intervals_kh = ...
243 [integer_intervals_kh1, ...
244 integer_intervals_kh2, ...
245 ];
246
247
248 unique_intervals_kh = unique(sort(sum_intervals_kh));
249
250 Cov_kh2 = length(unique_intervals_kh) / L * 100; 첫 번째, 두 번째 위성에 의해 얻을 수 있는
251 전체 관측 시간 동안의 커버리지 비율(서울)
252

```

2-6. 추출한 random_number로 두 번째 위성의 상대위치 결정 (블라디보스토크)

```

252 %% E_renewal 2
253 integer_intervals_eh1 = []; 2-3 에서 진행한 것과 동일하게 반복
254 integer_intervals_eh2 = [];
255
256 for i = 1:length(ii_0)
257 % 고각도 satellite 1 위치 생성
258 interval_eh1 = start_points4(i) :end_points4(i) ;
259 interval_eh1(interval_eh1 > (L-1)) = interval_eh1(interval_eh1 > (L-1)) - L;
260 integer_intervals_eh1 = [integer_intervals_eh1, interval_eh1];
261
262 % 고각도 satellite 2 위치 생성 2-4 에서 추출한 random_number를 추가하여 두 번째 위성의 위치를 결정
263 interval_eh2 = interval_eh1 + random_number_1;
264 interval_eh2(interval_eh2 > (L-1)) = interval_eh2(interval_eh2 > (L-1)) - L;
265 integer_intervals_eh2 = [integer_intervals_eh2, interval_eh2];
266 end
267
268
269 sum_intervals_eh = ...
270 [integer_intervals_eh1, ...
271 integer_intervals_eh2, ...
272 ];
273
274
275 unique_intervals_eh = unique(sort(sum_intervals_eh));
276
277 Cov_eh2 = length(unique_intervals_eh) / L * 100; 첫 번째, 두 번째 위성에 의해 얻을 수 있는 전체
278 관측 시간 동안의 커버리지 비율(블라디보스토크)

```


2.7. 2-2 ~ 2-6 과정을 반복하여 원하는 위성 수만큼 추가

```
편집기 - C:\Users\WACL2022_13\Desktop\학위 논문 연구 결과 자료_이진진\1.석사논문\최종 code\BILP_H10_L8.m
BILP_H10_L8.m
280 %% K_Satellite Setting(H3 / L0) - 앞에서 배치한 두 위성의 위치를 바탕으로 고각도 위성 1대
281
282 for n = 0:(L-1)
283     integer_intervals_kh1 = [] 2-2~2-6 과정을 반복하여 원하는 위성 수만큼 추가
284     integer_intervals_kh2 = [],
285     integer_intervals_kh3 = [];
286
287
288     for i = 1:length(gg_0)
289         % 고각도 satellite 1 위치 생성
290         interval_kh1 = start_points2(i) : end_points2(i) ;
291         interval_kh1(interval_kh1 > (L-1)) = interval_kh1(interval_kh1 > (L-1)) - L;
292         integer_intervals_kh1 = [integer_intervals_kh1, interval_kh1];
293
294         % 고각도 satellite 2 위치 생성
295         interval_kh2 = interval_kh1 + random_number_1;
296         interval_kh2(interval_kh2 > (L-1)) = interval_kh2(interval_kh2 > (L-1)) - L;
297         integer_intervals_kh2 = [integer_intervals_kh2, interval_kh2];
298
299         % 고각도 satellite 3 랜덤 위치 생성
300         interval_kh3 = interval_kh1 + n; 세 번째 위성 배치를 위한 무작위 n을 설정
301         interval_kh3(interval_kh3 > (L-1)) = interval_kh3(interval_kh3 > (L-1)) - L;
302         integer_intervals_kh3 = [integer_intervals_kh3, interval_kh3];
303     end
304
305
306     sum_intervals_kh = ...
307         [integer_intervals_kh1, ...
308         integer_intervals_kh2, ...
309         integer_intervals_kh3, ...
310         ];
311
312
313     unique_intervals_kh = unique(sort(sum_intervals_kh));
314
315     difference_kh = length(sum_intervals_kh) - length(unique_intervals_kh) ;
316
317     result_kh2 = [result_kh2; n, difference_kh];
318
319
```

3. 같은 방법으로 2-2 ~ 2-7 과정을 반복하여 높은 경사각의 궤도를 가진 위성 배치

```

2382 %% K_Satellite Setting(H10 / L1) - 고각도 위성의 위치를 바탕으로 저각도 위성 1대 배치(총 1
2383
2384 for n = 0:(L-1)
2385     integer_intervals_kh1 = [];
2386     integer_intervals_kh2 = [];
2387     integer_intervals_kh3 = [];
2388     integer_intervals_kh4 = [];
2389     integer_intervals_kh5 = [];
2390     integer_intervals_kh6 = [];
2391     integer_intervals_kh7 = [];
2392     integer_intervals_kh8 = [];
2393     integer_intervals_kh9 = [];
2394     integer_intervals_kh10 = [];
2395
2396     integer_intervals_kl0 = [];
2397     integer_intervals_kl1 = [];
2398
2399     for i = 1:length(gg_0)
2400         % 고각도 satellite 1 위치 생성
2401         interval_kh1 = start_points2(i) : end_points2(i) ;
2402         interval_kh1(interval_kh1 > (L-1)) = interval_kh1(interval_kh1 > (L-1)) - L;
2403         integer_intervals_kh1 = [integer_intervals_kh1, interval_kh1];
2404
2405         % 고각도 satellite 2 위치 생성
2406         interval_kh2 = interval_kh1 + random_number_1;
2407         interval_kh2(interval_kh2 > (L-1)) = interval_kh2(interval_kh2 > (L-1)) - L;
2408         integer_intervals_kh2 = [integer_intervals_kh2, interval_kh2];
2409
2410         .
2411         .
2412         .
2413
2414         % 고각도 satellite 9 위치 생성
2415         interval_kh9 = interval_kh1 + random_number_8;
2416         interval_kh9(interval_kh9 > (L-1)) = interval_kh9(interval_kh9 > (L-1)) - L;
2417         integer_intervals_kh9 = [integer_intervals_kh9, interval_kh9];
2418
2419         % 고각도 satellite 10 위치 생성
2420         interval_kh10 = interval_kh1 + random_number_9;
2421         interval_kh10(interval_kh10 > (L-1)) = interval_kh10(interval_kh10 > (L-1)) - L;
2422         integer_intervals_kh10 = [integer_intervals_kh10, interval_kh10];
2423     end
2424
2425     % 높은 궤도 경사각을 가진 위성 10대 배치(서울)
2426
2427     % 높은 궤도 경사각을 가진 위성 1대 배치(서울)
2428
2429     for i = 1:length(ff_0)
2430         % 저각도 satellite 0 위치 생성 기준 위성(시작 및 종료 지점만 참고하기 위한 용도)
2431         interval_kl0 = start_points1(i) : end_points1(i) ;
2432         interval_kl0(interval_kl0 > (L-1)) = interval_kl0(interval_kl0 > (L-1)) - L;
2433         integer_intervals_kl0 = [integer_intervals_kl0, interval_kl0];
2434
2435         % 저각도 satellite 1 위치 생성 첫 번째 위성(기준 위성 중심으로 위성 배치 시작)
2436         interval_kl1 = interval_kl0 + n;
2437         interval_kl1(interval_kl1 > (L-1)) = interval_kl1(interval_kl1 > (L-1)) - L;
2438         integer_intervals_kl1 = [integer_intervals_kl1, interval_kl1];
2439     end
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461

```

4. 모든 위성의 n의 값과 최종 커버리지 결과 출력

```
편집기 - C:\Users\WACL2022_13\Desktop\위성 위치 연구 결과 자료_이진진\1.석사논문\최종 code\BILP_H10_L8.m
BILP_H10_L8.m
%% 랜덤으로 배치한 모든 위성의 n을 출력
random_N(1) = (0);
random_N(2) = (random_number_1);
random_N(3) = (random_number_2);
random_N(4) = (random_number_3);
random_N(5) = (random_number_4);
random_N(6) = (random_number_5);
random_N(7) = (random_number_6);
random_N(8) = (random_number_7);
random_N(9) = (random_number_8);
random_N(10) = (random_number_9);
random_N(11) = (random_number_10);
random_N(12) = (random_number_11);
random_N(13) = (random_number_12);
random_N(14) = (random_number_13);
random_N(15) = (random_number_14);
random_N(16) = (random_number_15);
random_N(17) = (random_number_16);
random_N(18) = (random_number_17);

% 높은 궤도 경사각을 가진 위성 10대
% 낮은 궤도 경사각을 가진 위성 8대

% 사용자가 원하는 커버리지 비율 설정
percentage = 90;

Cov_kh10_8 서울 커버리지 비율(높은 궤도 경사각 위성 10대, 낮은 궤도 경사각 위성 8대)
Cov_eh10_8 블라디보스토크 커버리지 비율

% 고각도 위성 및 저각도 위성의 수 입력 (최소 1부터)
H0 = 10;
L0 = 8;

% 두 지역에 대한 커버리지를 모두 달성하기 위한 조건 설정
if Cov_kh10_8 >= percentage && Cov_eh10_8 >= percentage
    break;
end
trials = trials + 1;
end

% 시뮬레이션 종료 시간 기록
elapsed_time = toc(start_time); 시뮬레이션 소요시간 출력(초)

% 결과 출력
fprintf('시행 횟수: %d\n', trials);
fprintf('결린 시간(초): %.2f\n', elapsed_time);

% 변수 목록
vars = who; vars 함수를 사용하여 위성 배치를 추가한 만큼 위성 수 N을 결정

% 'Cov_eh'로 시작하는 변수들만 필터링
pattern = '^Cov_eh\d+$'; % 정규 표현식 패턴
N = 0;

for i = 1:length(vars)
    if ~isempty(regexp(vars{i}, pattern, 'once'))
        N = N + 1;
    end
end
```

5. 그래프 기본 정보 설정 (ex. 서울을 지나는 높은 궤도경사각을 가진 위성)

```
편집기 - C:\Users\ACL2022_13\Desktop\학위 논문 연구 결과 자료_이진진\1 석사논문\최종 code\BILP_H10_L8.m
BILP_H10_L8.m
6615 %% 고각도 그래프 Seoul
6616
6617 n_1 = random_N(1:H0) +1; 고각도 위성 10대에 대한 n을 n_1에 저장
6618 (기준 위성을 첫번째 위성으로 지정하였으므로 첫번째 n은 항상 0)
6619 for i = 1:length(n_1)
6620     eval(['gg_', num2str(i), ' = gg_0 + n_1(' , num2str(i), ');']);
6621 end
6622 % 각각의 n을 더한 값에 따라 달라지는 위성 10대(length(n_1))의
6623 % x축 범위 설정 커버리지 시작 및 종료 시각을 g_1~g_10으로 지정
6624 x_range = 0:L-1; 그래프 출력에 위한 x축의 범위를 설정(0~719까지의 정수, 총 720개)
6625
6626 for idx = 1:length(n_1)
6627     gg = eval(['gg_', num2str(idx)]); 위에서 지정한(Line 6619~6621) n의 값에 따른 위성
6628     x_coords1{idx} = []; 커버리지의 시작 및 종료 시각을 1*10의 cell에 저장함
6629
6630     for i = 1:size(gg,1)
6631         start_time = gg(i, 1); 고각도 궤도의 모든 위성의 커버리지 시작 및 종료 시각을
6632         end_time = gg(i, 2); start_time과 end_time으로 지정
6633
6634         % y=1일 때의 시작 시간과 종료 시간의 x좌표 계산
6635         if end_time > L-1 && start_time < L
6636             start_index = find(x_range >= start_time, 1);
6637             end_index = find(x_range < end_time-L, 1, 'last');
6638         elseif start_time > L-1
6639             start_index = find(x_range >= start_time-L, 1);
6640             end_index = find(x_range <= end_time-L, 1, 'last');
6641         else
6642             start_index = find(x_range >= start_time, 1);
6643             end_index = find(x_range <= end_time, 1, 'last');
6644         end
6645         위에서 지정한 n의 값이 관측시간의 총 길이(720)를 초과하는 경우,
6646         초과한 만큼이 처음부터 되돌아오도록 보정
6647
6648         % 시작 시간부터 종료 시간까지의 x좌표를 배열에 저장
6649         if end_time > L-1 && start_time < L
6650             x_coords1{idx} = [x_coords1{idx}, x_range(start_index:L),(0:end_index)];
6651         else
6652             x_coords1{idx} = [x_coords1{idx}, x_range(start_index:end_index)];
6653         end
6654         보정한 값을 x_coords1에 재배치
6655
6656     end
6657 end
6658
6659
6660
```



```

6661 % 모든 작업공간의 x 좌표를 모은 배열 초기화
6662 all_x_coords1 = []; all_x_coords1의 배열을 초기화
6663
6664 %% 각 작업공간의 x 좌표를 all_x_coords1에 추가하고 작은 수부터 오름차순으로 정렬
6665 for idx = 1:length(n_1)
6666     all_x_coords1 = [all_x_coords1, x_coords1{idx}];
6667 end
6668 x_coords1에 배치된 값들을 all_x_coords1에 double 형태로 재배치
6669 all_x_coords1 = sort(all_x_coords1); all_x_coords1을 오름차순으로 정렬
6670
6671 % 중복된 값을 제거하고 유일한 값들만 추출 중복된 값을 제거하여 관심지역(서울)을 지나는
6672 unique_x_coords1 = unique(all_x_coords1); 위성의 시각(n)을 unique_x_coords1에 저장
6673
6674 coverage1 = length(unique_x_coords1) / L * 100; 전체 관측 시간 중 위성이 관심지역(서울)을 지나는 비율을
6675 coverage1에 저장
6676
6677 %%
6678 % 초기화
6679 RV1 = []; RV의 배열을 초기화
6680 current_value = all_x_coords1(1); 커버리지의 중복이 포함된 all_x_coords1을 current_value에 저장
6681 start_index = 1;
6682
6683 % 각 원소에 대해 반복하면서 연속된 값의 구간을 찾을
6684 for i = 2:length(all_x_coords1)
6685     if all_x_coords1(i) ~= current_value
6686         % 현재 값과 다른 값이 나타난 경우, 해당 구간의 시작과 끝을 저장하고 현재 값 갱신
6687         end_index = i - 1;
6688         RV1 = [RV1; current_value, end_index - start_index + 1];
6689         current_value = all_x_coords1(i);
6690         start_index = i; current_value에서 앞뒤로 중복된 값이 나오는 경우를 제외하여 RV1에 저장
6691     end
6692 end
6693
6694 % 마지막 구간에 대한 처리
6695 end_index = length(all_x_coords1); 마지막 구간을 처리 후, 각 n에 따른 접근 위성의 수를 RV1에 저장
6696 RV1 = [RV1; current_value, end_index - start_index + 1];
6697
6698 %% 빠진 숫자 찾기
6699
6700 % 주어진 행렬 접근 위성의 수가 반영된 RV1 행렬을 matrix1으로 지정 후 0부터 719까지의 정수 중
6701 matrix1 = RV1; 누락된 숫자를 missing_numbers에 저장
6702
6703 % 현재 1열에서 빠진 숫자를 찾기
6704 missing_numbers = setdiff(0:L-1, matrix1(:,1));
6705
6706 % 새롭게 추가된 행에 대해 0으로 입력
6707 new_rows = zeros(length(missing_numbers), 2); missing_numbers에서 찾은 숫자를 1열에 배치하고
6708 new_rows(:, 1) = missing_numbers; 2열에는 0으로하여 새로운 new_rows행렬을 생성
6709
6710 % 새로운 행렬 생성
6711 new_matrix1 = [matrix1; new_rows]; 기존의 matrix1과 새롭게 생성한 new_rows를 합산하여 new_matrix1에 저장
6712 (0부터 719까지의 모든 구간에서 접근 위성수를 확인 가능)
6713
6714 % 1열을 기준으로 행렬을 정렬
6715 sorted_matrix1 = sortrows(new_matrix1, 1); new_matrix1을 오름차순으로 정렬
6716
6717

```

6-1. 기준 위성의 Access Profile 그래프 정보 설정 (ex. 서울, 고각도 위성)

```

6718 %% Access Profile
6719
6720 % x축 범위 설정
6721 x_range = 0:L-1;
6722
6723 figure;
6724 subplot(3, 1, 1);
6725
6726 hex_color1 = '#cc4e01';
6727
6728 red1 = hex2dec(hex_color1(2:3)) / 255; % R 값
6729 green1 = hex2dec(hex_color1(4:5)) / 255; % G 값
6730 blue1 = hex2dec(hex_color1(6:7)) / 255; % B 값
6731
6732 hex_color11 = '#fe9882';
6733
6734 red11 = hex2dec(hex_color11(2:3)) / 255; % R 값
6735 green11 = hex2dec(hex_color11(4:5)) / 255; % G 값
6736 blue11 = hex2dec(hex_color11(6:7)) / 255; % B 값
6737
6738
6739 % 각 구간별로 y=1인 위치 표시
6740 for i = 1:size(gg_0, 1)
6741     if neg1(i) < 0
6742         start_time_new = L + neg1(i,1);
6743         end_time_new = L + neg1(i,2);
6744     else
6745         start_time_new = gg_0(i, 1);
6746         end_time_new = gg_0(i, 2);
6747     end
6748     start_time = gg_0(i, 1);
6749     end_time = gg_0(i, 2);
6750
6751     if end_time_new > L-1
6752         end_time_new = L-1;
6753     end
6754
6755 % 시작 시간과 종료 시간에 해당하는 x 인덱스 찾기
6756 start_index = find(x_range >= start_time, 1);
6757 end_index = find(x_range <= end_time, 1, 'last');
6758 stn = start_time_new;
6759 etn = end_time_new;
6760
6761 % y 값 설정
6762 y_values1a = zeros(size(x_range));
6763 y_values1a(start_index:end_index) = 1;
6764 if neg1(i) < 0
6765     y_values1a(stn:etn) = 1;
6766 end
6767
6768 % 그래프 그리기
6769 plot(x_range, y_values1a, '-o n', 'LineWidth', 1, 'MarkerSize', 1, 'Color', [red1, green1, blue1])
6770 hold on; 그래프 아래 면적 색깔
6771 area(x_range, y_values1a, 'FaceColor', [red11, green11, blue11]);
6772 plot(x_range, y_values1a, 'Color', [red1, green1, blue1], 'LineWidth', 1.5);
6773 end 선 색깔
6774
6775 % 그래프 세부 설정
6776 xlabel('n');
6777 ylabel('v0j[n]');
6778 title('Access Profile');
6779 grid on;
6780 xlim([0, L-1]);
6781 ylim([0, 1]);
6782

```

그래프의 색상 지정

그래프가 관측 시간의 총 길이를 초과하는 경우,
처음으로 돌아오도록 보정

기준 위성의 커버리지 정보만
포함되도록 y_values1a를 설정

선 두께, 색상 및 투명도 등 그래프 정보 상세 설정

6-2. Constellation Pattern Vector 그래프 정보 설정 (ex. 서울, 고각도 위성)

```
편집기 - C:\Users\ACL2022_13\Desktop\학위 논문 연구 결과 자료_이진진\1.석사논문\최종 code\BILP_H10_L8.m
BILP_H10_L8.m
6783 %% Constellation Pattern Vector
6784
6785 % x_range = 0:L-1;
6786
6787 subplot(3, 1, 2);
6788
6789 % y 값 설정 모든 위성의 n의 정보가 포함되도록 y_values1p를 설정
6790 y_values1p = zeros(size(x_range));
6791 y_values1p(n_1(1,:)) = 1;
6792
6793
6794
6795 plot(x_range, y_values1p, 'Color', [red1, green1, blue1], 'LineWidth', 1, 'MarkerSize', 1);
6796 title('Constellation Pattern Vector');
6797 hold on;
6798
6799 xlim([0, L-1]);
6800 ylim([0, 1]);
6801
6802 % 그래프 세부 설정
6803 xlabel('n');
6804 ylabel('x[n]');
6805 grid on;
6806 xlim([0, L-1]);
6807 ylim([0, 1]);
6808
```

6-3. Coverage Timeline 그래프 정보 설정 (ex. 서울, 고각도 위성)

```
편집기 - C:\Users\ACL2022_13\Desktop\학위 논문 연구 결과 자료_이진진\1.석사논문\최종 code\BILP_H10_L8.m
BILP_H10_L8.m
6810 %% Coverage Timeline
6811
6812 x_range = 0:L-1;
6813
6814 subplot(3, 1, 3);
6815 plot(sorted_matrix1(:,1), sorted_matrix1(:,2), 'Color', [red1, green1, blue1], 'LineWidth', 1,
6816 title('Coverage Timeline');
6817 hold on;
6818 area(sorted_matrix1(:,1), sorted_matrix1(:,2), 'FaceColor', [red11, green11, blue11]);
6819 plot(sorted_matrix1(:,1), sorted_matrix1(:,2), 'Color', [red1, green1, blue1], 'LineWidth', 1);
6820
6821 xlim([0, L-1]);
6822 ylim([0, 3]);
6823
6824 % 그래프 세부 설정
6825 xlabel('n');
6826 ylabel('bj[n]');
6827 grid on;
6828 xlim([0, L-1]);
6829
```

같은 방법으로 그래프 설정

서울 저각도(Line 6833~7049)

블라디보스토크 고각도(Line 7201~7415)

블라디보스토크 저각도(Line 7420~7637)

그래프 결과

figure 1 : 높은 궤도 경사각을 가진 위성 10대를 운용했을 때 서울의 커버리지

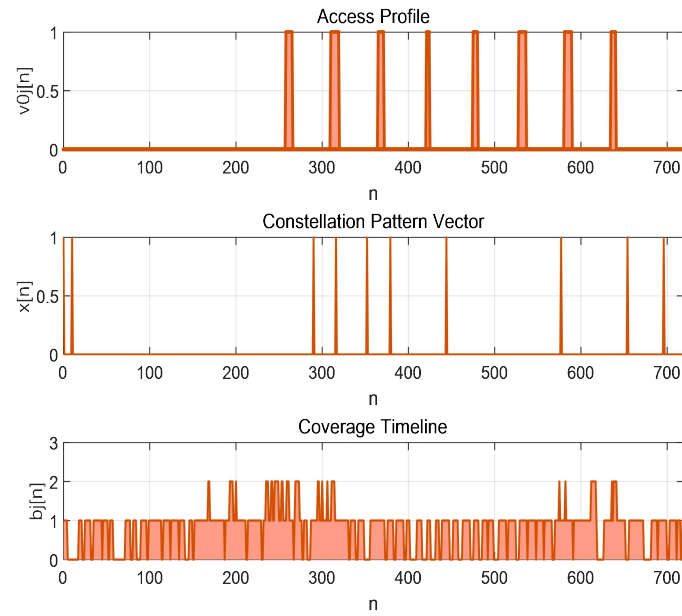


figure 2 : 낮은 궤도 경사각을 가진 위성 8대를 운용했을 때 서울의 커버리지

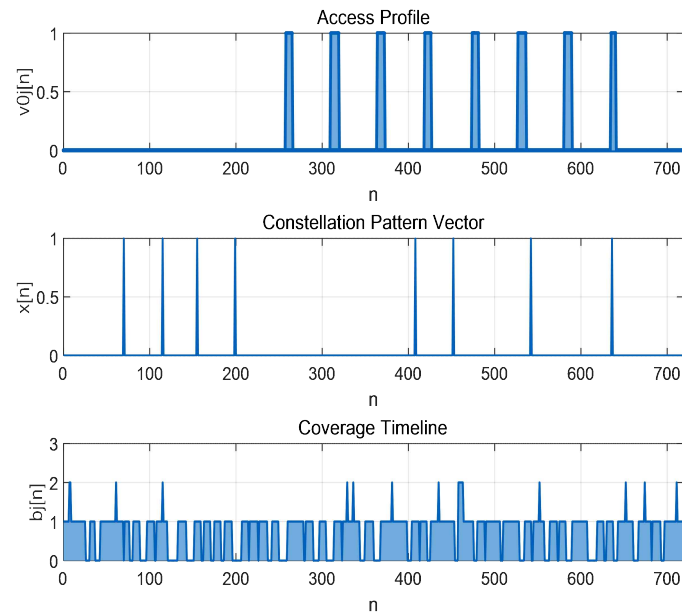


figure 4 : 높은 궤도 경사각을 가진 위성 10대를 운용했을 때 블라디보스토크의 커버리지

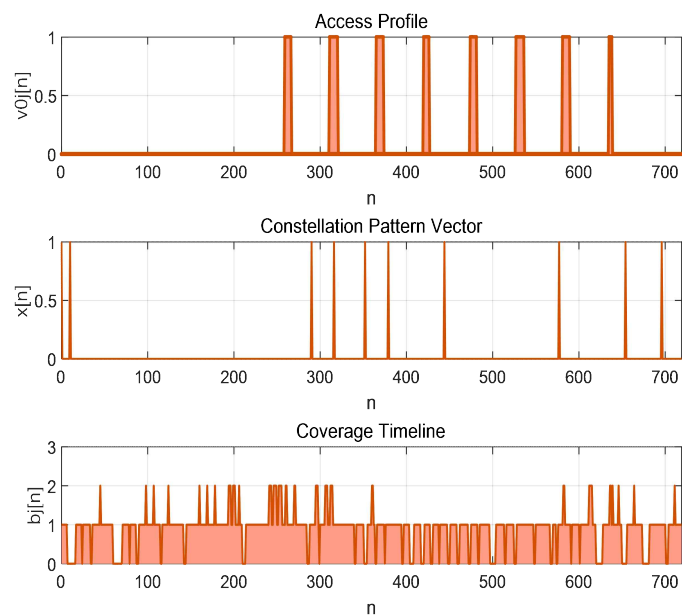


figure 5 : 낮은 궤도 경사각을 가진 위성 8대를 운용했을 때 블라디보스토크의 커버리지

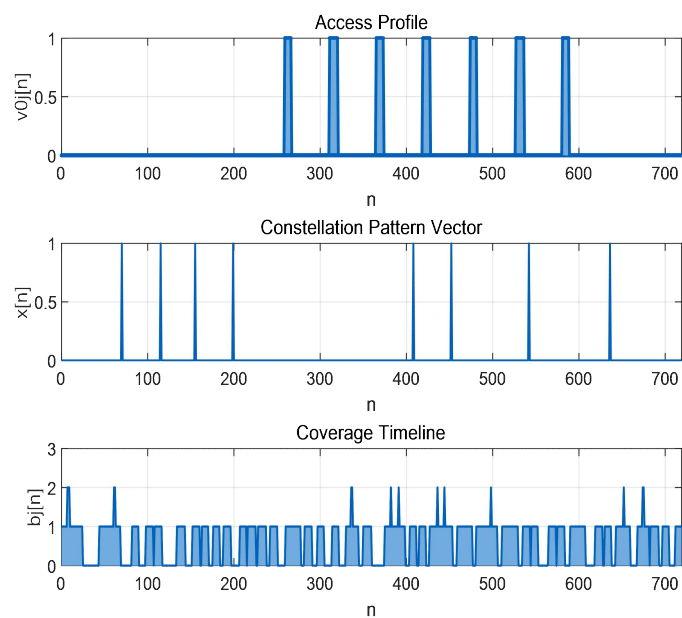


figure 3 : 높은 궤도 경사각을 가진 위성 10대, 낮은 궤도 경사각을 가진 위성 8대
총 18대의 위성을 운용했을 때 서울의 커버리지(figure 1과 2를 겹침)

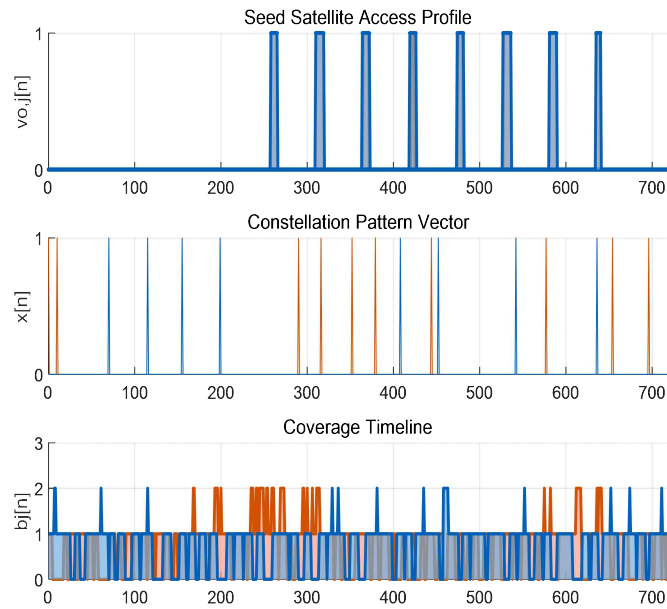


figure 6 : 높은 궤도 경사각을 가진 위성 10대, 낮은 궤도 경사각을 가진 위성 8대
총 18대의 위성을 운용했을 때 블라디보스토크의 커버리지(figure 4과 5를 겹침)

